Министерство науки и высшего образования Российской Федерации
Калужский филиал
федерального государственного автономного
образовательного учреждения высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(КФ МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ***ИУК «Информатика и управление»***

КАФЕДРА ***ИУК4 «Программное обеспечение ЭВМ, информационные технологии»***

ЛАБОРАТОРНАЯ РАБОТА №2

«Журнализация изменений в СУБД PostgreSQL»

ДИСЦИПЛИНА: «Проектирование систем управления базами данных»

Выполнил: студент гр. ИУК4-41М _____ (Сафронов Н.С.)
(Подпись) (Ф.И.О.)

Проверил: _____ (Глебов Ю.С.)
(Подпись) (Ф.И.О.)

Дата сдачи (защиты):

Результаты сдачи (защиты):

- Балльная оценка:
- Оценка:

Калуга, 2026

Цель: формирование практических навыков работы с журналированием баз данных.

Задание

Вариант 2

1. Создайте таблицы из вашего варианта в лабораторной работе №1.

2. Определите, сколько занимает таблица

а) страниц на диске,

б) буферов в кэше.

3. Сколько байтов занимают сгенерированные журнальные записи?

Чем можно объяснить довольно большое число?

Результат выполнения работы:

```
test_project=# \d+ __orbismap3.__files
                                                  Table "__orbismap3.__files"
    Column     |            Type             | Collation | Nullable |     Default     | Storage  | Compression | Stats target | Description
---------------+-----------------------------+-----------+----------+-----------------+----------+-------------+--------------+-------------
 group_id      | character varying(32)       |           | not null |                 | extended |             |              |
 file_hash     | character varying(32)       |           | not null |                 | extended |             |              |
 file_path     | text                        |           | not null |                 | extended |             |              |
 ext           | character varying(32)       |           |          |                 | extended |             |              |
 attrs         | jsonb                       |           |          | '{}'::jsonb     | extended |             |              |
 file_size     | bigint                      |           | not null | 0               | plain    |             |              |
 user_id       | integer                     |           |          |                 | plain    |             |              |
 updated_at    | timestamp without time zone |           | not null |                 | plain    |             |              |
 module        | character varying(32)       |           |          |                 | extended |             |              |
 map_id        | oid                         |           |          |                 | plain    |             |              |
 container_oid | oid                         |           |          |                 | plain    |             |              |
 delete_at     | timestamp without time zone |           |          |                 | plain    |             |              |
 tags          | text[]                      |           |          | ARRAY[]::text[] | extended |             |              |
 storage_id    | character varying(32)       |           |          |                 | extended |             |              |
 root_path     | text                        |           | not null | '.'::text       | extended |             |              |
 confirmed     | boolean                     |           | not null | false           | plain    |             |              |
 public_file   | boolean                     |           | not null | false           | plain    |             |              |
Indexes:
    "__files_pkey" PRIMARY KEY, btree (group_id, file_hash)
    "files_tags_idx" gin (tags)
    "files_target_idx" btree (map_id, container_oid)
    "idx_storage_root_hash_unique" UNIQUE, btree (storage_id, md5(root_path), file_hash)
Check constraints:
    "__files_attrs_check" CHECK (starts_with(attrs::text, '{'::text))
Access method: heap

test_project=# SELECT COUNT(1) FROM __orbismap3.__files;
 count
-------
    29
(1 row)

test_project=#
```

Рисунок 1 – Пример таблицы, над которой будет осуществлять дальнейшая работа

```
test_project=# SELECT pg_relation_size('__orbismap3.__files') AS size_bytes,
test_project-# pg_relation_size('__orbismap3.__files') / current_setting('block_size')::INT AS pages,
test_project-# pg_size_pretty(pg_relation_size('__orbismap3.__files')) AS pretty_size;
 size_bytes | pages | pretty_size
------------+-------+-------------
     196608 |    24 | 192 kB
(1 row)

test_project=#
```

Рисунок 2 – Получение размера, занимаемого таблицей

```
test_project=# CREATE EXTENSION pg_buffercache;
CREATE EXTENSION
test_project=# SELECT
    COUNT(1) AS buffers_in_cache,
    COUNT(1) * current_setting('block_size')::INT AS cached_bytes,
    pg_size_pretty(COUNT(1) * current_setting('block_size')::INT) AS pretty_cached
FROM pg_buffercache b
JOIN pg_catalog.pg_class c
    ON b.relfilenode = c.relfilenode
JOIN pg_catalog.pg_namespace n
    ON n.oid = c.relnamespace
WHERE
    c.relname = '__files'
    AND n.nspname = '__orbismap3'
    AND b.reldatabase = (SELECT oid FROM pg_catalog.pg_database WHERE datname = CURRENT_DATABASE());
 buffers_in_cache | cached_bytes | pretty_cached
------------------+--------------+---------------
               28 |       229376 | 224 kB
(1 row)

test_project=#
```

Рисунок 3 – Получение количества буферов в кэше и размера кэша для таблицы

```
test_project=# SELECT pg_current_wal_lsn() AS start_lsn;
 start_lsn
------------
 4/F90DF100
(1 row)

test_project=# DELETE FROM __orbismap3.__files WHERE file_hash = 'd09fb4e3739a44d6604cf0d535e509df';
DELETE 4
test_project=# SELECT pg_switch_wal();
 pg_switch_wal
---------------
 4/F90E0828
(1 row)

test_project=# SELECT pg_wal_lsn_diff(pg_current_wal_lsn(), '4/F90DF100') AS wal_bytes_generated;
 wal_bytes_generated
---------------------
            15863648
(1 row)

test_project=#
```

Рисунок 4 – Получение размера WAL-записей с изменениями и последующей проверкой изменения размера журнала

Чем можно объяснить довольно большое число?

Значительный объем сгенерированных журнальных записей WAL объясняется совокупностью нескольких факторов архитектуры PostgreSQL. Механизм Full Page Writes (FPW) предполагает запись полной копии страницы (8 КБ) в журнал при её первом изменении после контрольной точки, даже если модифицирован всего один байт, что необходимо для гарантированного восстановления после сбоя. Каждая WAL-запись несет служебную нагрузку в виде заголовка, идентификатора транзакции (XID), позиции (LSN) и контрольной суммы, что добавляет около 100–200 байт накладных расходов на каждую операцию. Кроме того, логированию подлежат не только изменения основных данных, но и модификации индексов (B-tree, GIN и др.), а также больших значений, вынесенных в TOAST-таблицы.

Вывод: в ходе выполнения лабораторной работы были получены навыки работы с журналированием баз данных.